Android 13 Camera
开发指南

版本号: 1.8
发布日期: 2025.5.26

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.1	2020.11.18	KPA0368	初始版本
1.2	2020.12.31	KPA0368	camera 配置文件独立，修改文件路径
1.3	2021.11.26	KPA0368	更改 USB Camera 相关的配置
1.4	2022.12.5	KPA0368	增加 debug 信息采集
1.5	2023.3.18	KPA0368	1. 增加差异化配置描述 2. 删除 Android Framework 描述
1.6	2024.3.15	KPA0368	1. 删减 FAQ 章节 2. 增加 Hal1 介绍 3. 新增章节索引 4. 合并选型指南部分至开发指南
1.7	2025.4.7	KPA0368	1. 新增 video 节点配置方式
1.8	2025.5.26	KPA0368	1. 新增 camera 框架运行简介

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 概述 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 相关人员 1

2 Camera 硬件链路 **2**

3 Android Camera 选型 **3**
3.1 Sensor 方向选择 3
3.2 GMS 方案 Camera Sensor 要求 4

4 Android 软件层次 **5**

5 Camera 框架运行简介 **7**

6 Camera HAL 介绍 **9**
6.1 Camera Hal1 框架 9
6.2 Camera Hal3 框架 10
6.2.1 camera_module 10
6.2.2 camera3_device_ops_t 12
6.2.2.1 initialize 12
6.2.2.2 configure_stream 12
6.2.2.3 construct_default_request_settings 14
6.2.3 数据流向 14
6.2.3.1 process_capture_request 14
6.2.3.2 process_capture_result 15

7 Camera 配置 **18**
7.1 内核配置介绍 18
7.2 Sensor 自适应配置 18
7.3 HAL 选择 18
7.4 CSI Camera 配置 18
7.4.1 配置文件介绍 18
7.4.2 Camera 方向配置 19
7.4.3 数字变焦功能配置 19
7.4.4 单摄配置 19
7.4.5 插值方案配置 20
7.4.6 闪光灯 20
7.4.7 对焦功能配置 21
7.4.8 Camera 节点配置 22

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7.5 USB Camera 配置 22
7.5.1 内核配置 22
7.5.2 Andoird 配置 23

8 常见问题排查 25

广州星翼电子科技有限公司zdyzlinux

插　图

图 2-1　Camera 硬件结构图 2
图 3-1　横屏与竖屏差别图 3
图 3-2　成像方向图 3
图 3-3　横屏成像示意图 4
图 3-4　竖屏成像示意图 4
图 4-1　Android 软件层次 5
图 5-1　camera_model 7
图 6-1　hal1 框架图 9
图 6-2　V4L2CameraHAL 接口流程图 11
图 6-3　camera3_device_ops_t-initialize 流程图 12
图 6-4　configure_stream 13
图 6-5　process_capture_request 15
图 6-6　process_capture_result 17
图 7-1　sensor 驱动检查 21

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 概述

1.1 编写目的

本文旨在介绍Camera 硬件链路,Android Camera 选型，Android Camera 软件层次 , HAL 代码流程，Android Camera 功能配置，方便客户快速上手全志平台开发。

1.2 适用范围

本文档适用于 Android Camera 系统开发。

1.3 相关人员

Camera 相关的 HAL，驱动以及应用开发人员。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 Camera 硬件链路

Camera 模块的硬件层次如下图所示, 被拍摄的景物通过镜头 (lens) 将生成的光学图像投射到传感器 (sensor) 上。传感器的作用是将采集到的光信号转换为电信号，并通过模数转换 (A/D) 转化为数字信号。图像的数字信号通过 CSI 接口存储到 DRAM 中。然后 ISP(Image Signal Processor) 对图像进行包括噪音消除，暗黑补偿，3A 处理，色彩空间变换等一系列处理后生成 YUV 图像，最后可以送到 VE 编码器进行图像视频的编码，或者送到 DE 显示引擎进行显示。

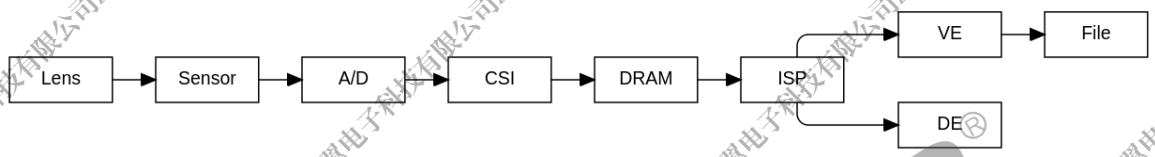

图 2-1: Camera 硬件结构图

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 Android Camera 选型

3.1 Sensor 方向选择

对物理横屏（如分辨率：1280x800）和物理竖屏（如分辨率：800x1280），对摄像头成像角度是有不同的要求的。

横屏与竖屏差别（下面左边是物理横屏，右边是物理竖屏）：

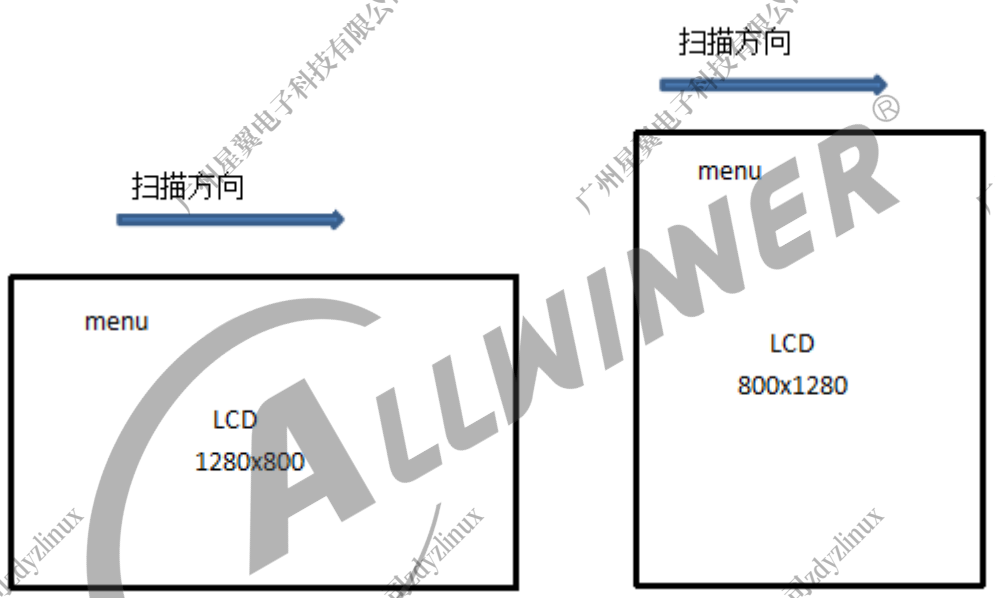

图 3-1: 横屏与竖屏差别图

摄像头成像方向有如下两种（左边为 0 度，右边为 90 或 270 度）：

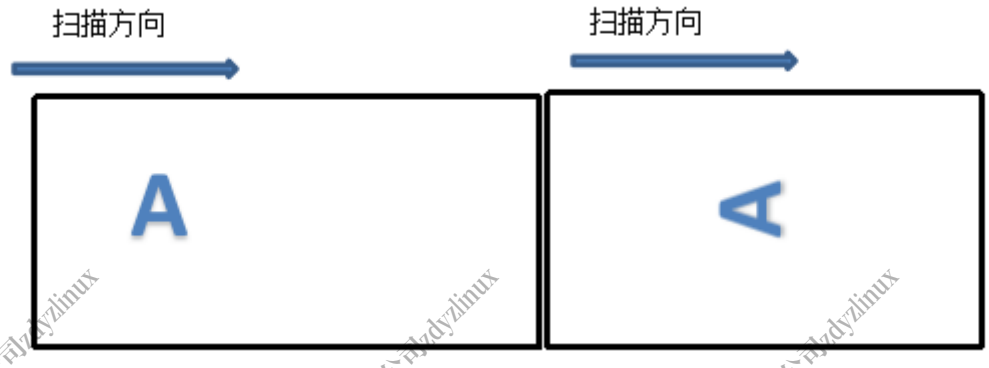

图 3-2: 成像方向图

版权所有 © 珠海全志科技股份有限公司。保留一切权利

正确的配对方式为：横屏时应选择摄像头成像角度为 0 度的模组。

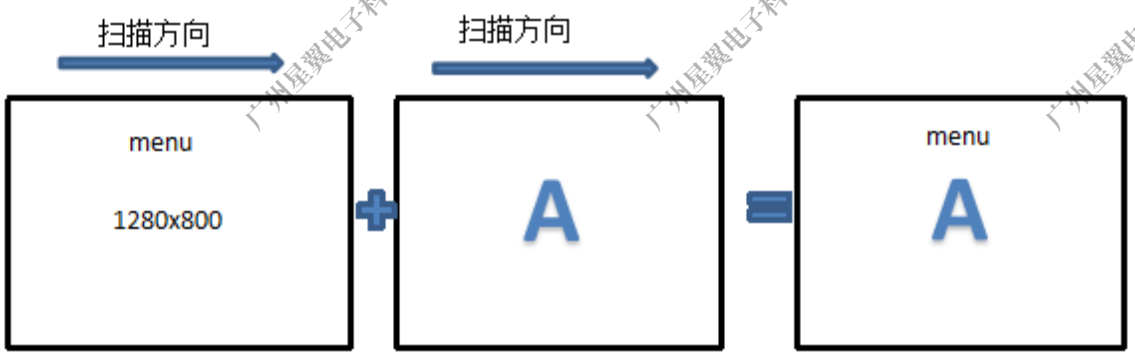

图 3-3: 横屏成像示意图

竖屏竖屏时应选择摄像头成像角度为 90 度或者 270 度的模组

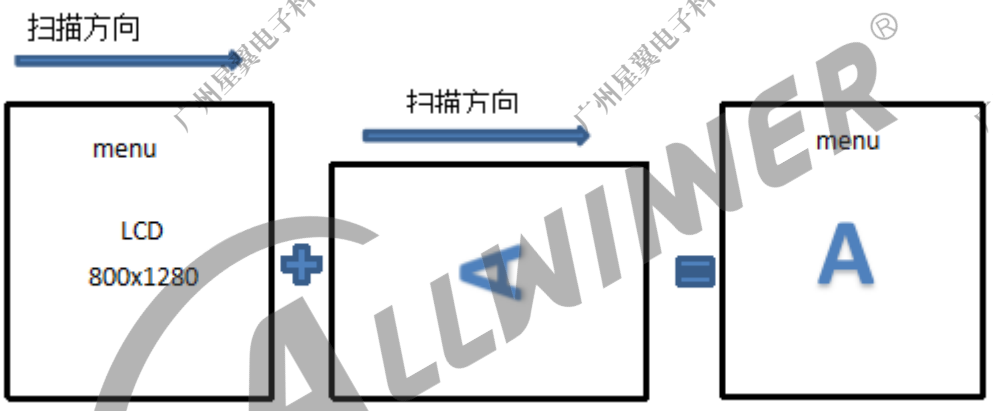

图 3-4: 竖屏成像示意图

3.2 GMS 方案 Camera Sensor 要求

对于 GMS 方案，谷歌对 Sensor 的分辨率有最低限制要求，可登录Android CDD 官网查询。

说明

对于 GMS 方案，还需注意必须前后摄像头需满足要求，如：
后置摄像头要求：分辨率宽 x 高的像素必须大于等于 200 万像素，否则 GMS camera 模块 android.hardware.camera2.cts.ExtendedCameraCharacteristicsTest#testAvailableStreamConfigs 这一项会过不了测试。因此在 Camera 模组选型的时候后置摄像头的分辨率宽 x 高的像素必须大于等于 200 万像素。
前置摄像头要求：谷歌最新的 CDD 要求如下：MUST have a resolution of at least VGA (640x480 pixels).

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 Android 软件层次

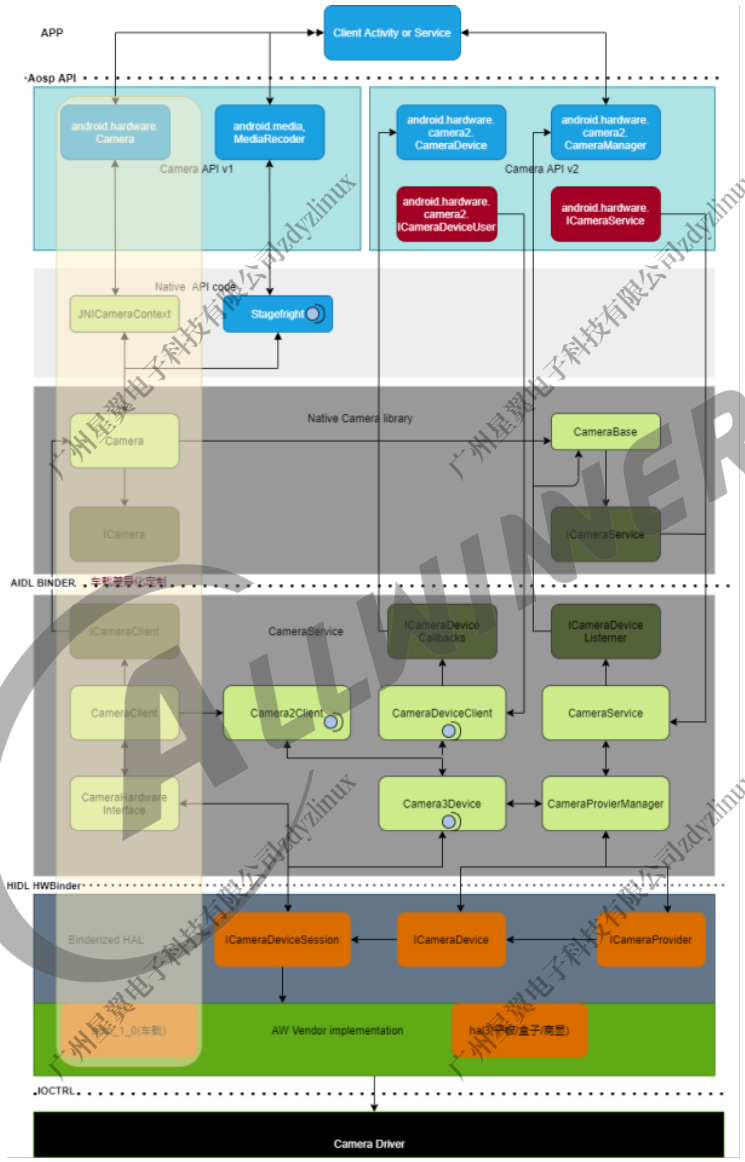

图 4-1: Android 软件层次

- 应用框架

应用代码位于应用框架级别，它使用 Camera 1/2 API 与相机硬件进行交互。在内部，这些代码会调用相应的 Binder 接口，以访问与相机互动的 Native 代码。

- Framework/AIDL 框架

版权所有 © 珠海全志科技股份有限公司。保留一切权利

与 CameraService 关联的 Binder 接口可在 frameworks/av/camera/aidl/android/hardware 中找到。生成的代码会调用较低级别的 Native 代码以获取对实体相机的访问权限，并返回用于在 Framework 级别创建 CameraDevice, 并最终创建 CameraCaptureSession 对象的数据。

- Native 框架

此框架位于 frameworks/av/ 中，并提供相当于 CameraDevice 和 CameraCaptureSession 类的原生类。

- Binder IPC 接口

IPC binder 接口用于实现跨越进程边界的通信。调用相机服务的若干个相机 Binder 类位于 frameworks/av/camera/camera/aidl/android/hardware 目录中。ICameraService 是相机服务的接口；ICameraDeviceUser 是已打开的特定相机设备的接口；ICameraServiceListener 和 ICameraDeviceCallbacks 分别是对应用框架的 CameraService 和 CameraDevice 回调。

- 相机服务

位于 frameworks/av/services/camera/libcameraservice/CameraService.cpp 下的相机服务是与 HAL 进行互动的实际代码。

- HAL

硬件抽象层定义了由相机服务调用、且必须实现以确保相机硬件正常运行的标准接口，相机 HAL 的 HIDL 接口在 hardware/interfaces/camera 中定义, 且典型的 Binderized HAL, 必须要实现如下接口:

- ICameraProvider：用于枚举单个设备并管理其状态。
- ICameraDevice：相机设备接口。
- ICameraDeviceSession：活跃的相机设备会话接口。

Hal1用于车载，Hal3则用于商显，平板，盒子类产品；如非车载方案，Hal1 部分介绍可以跳过。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 Camera 框架运行简介

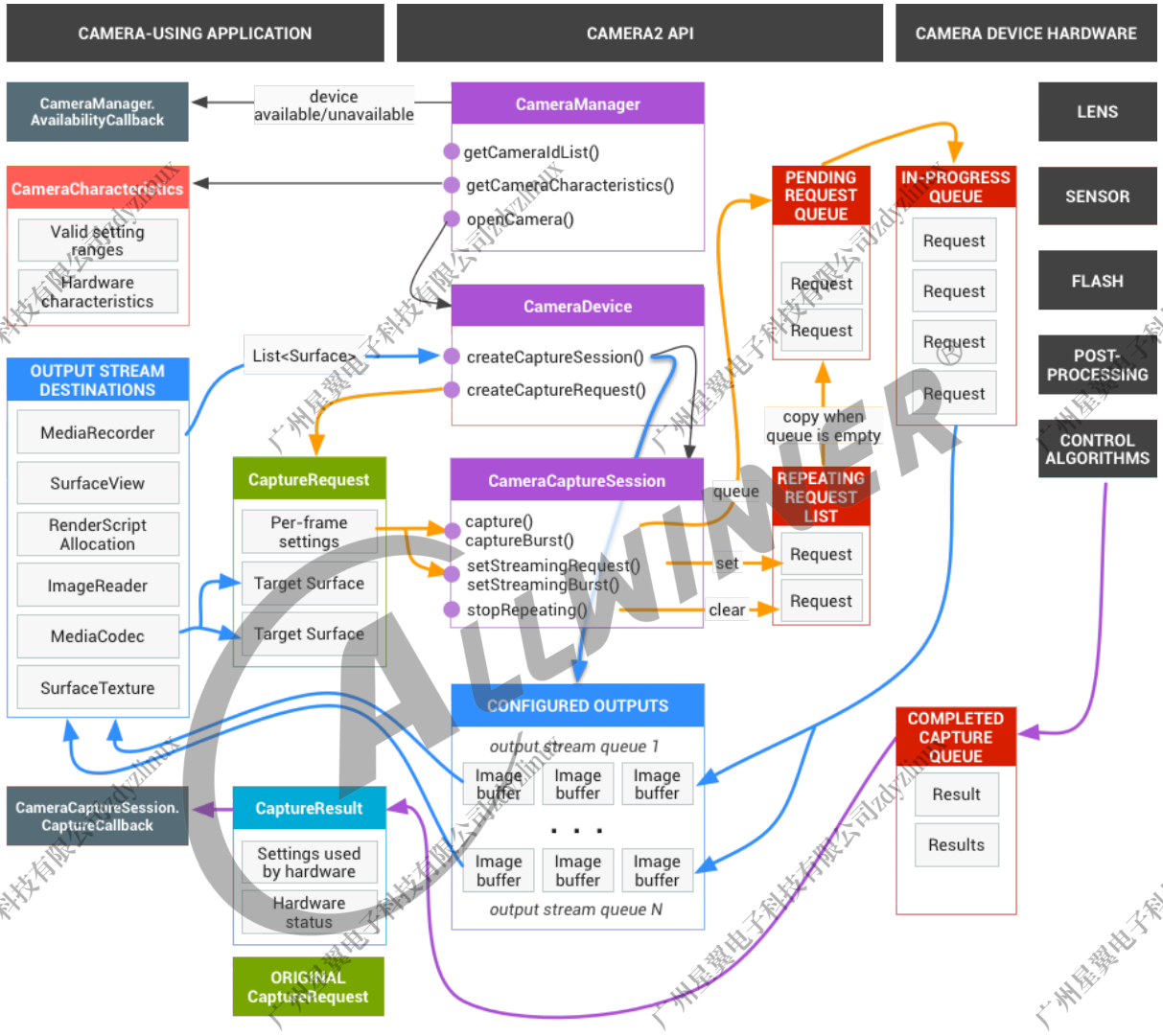

图 5-1: camera_model

1. App 一般在 MainActivity 中使用 SurfaceView 或者 SurfaceTexture+TextureView 或者 GLSurfaceView 等控件作为显示预览界面的控件, 共同点都是包含了一个单独的 Surface 作为取相机数据的容器.
2. APP 通过获取 Camera 服务 openCamera 成功后。会有回调从 CameraServer 通知到 App, 在 onOpenedCamera 回调中去调用类似 startPreview 的操作. 此时会创建 CameraCaptureSession, 创建过程中会向 CameraServer 调用 ConfigureStream 的操作,ConfigureStream 的参数中包含了第一步中空间中的 Surface 的引用, 相当于 App 将 Surface 容器给到了 CameraServer,CameraServer 包装了下该 Surface 容器为 stream, 通过 HIDL/ALDL 传递给 HAL, 继

版权所有 © 珠海全志科技股份有限公司。保留一切权利

而 HAL 也做 configureStream 操作.hal 层会根据流的配置，设置 camera 的出图 size 和 format, 并开流；

3. ConfigureStream 成功后 CameraServer 会给 App 回调通知 ConfigStream 成功, 接下来 App 便会调用 AOSP setRepeatingRequest 接口给到 CameraServer,CameraServer 初始化时便起来了一个死循环线程等待来接收 Request.
4. CameraServer 将 request 交到 Hal 层去处理,Hal 会与 V4L2 框架进交互以及运行数据流，并处理请求中的 3A 操作；得到 HAL 处理结果后取出该 Request 的处理 Result 中的 Buffer 填到 App 给到的容器中,SetRepeatingRequest 为了预览, 则交给 Preview 的 Surface 容器, 如果是 Capture Request 则将收到的 Buffer 交给 ImageReader 的 Surface 容器.
5. Surface 本质上是 BufferQueue 的使用者和封装者,BufferQueue 机制会通知到应用, 此时 App 中控件取出各自容器中的内容消费掉,Preview 控件中的 Surface 中的内容将通过 View 提供到 SurfaceFlinger 中进行合成最终显示出来, 即预览; 而 ImageReader 中的 Surface 被填了, 则 App 将会取出保存成图片文件消费掉；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 Camera HAL 介绍

6.1 Camera Hal1 框架

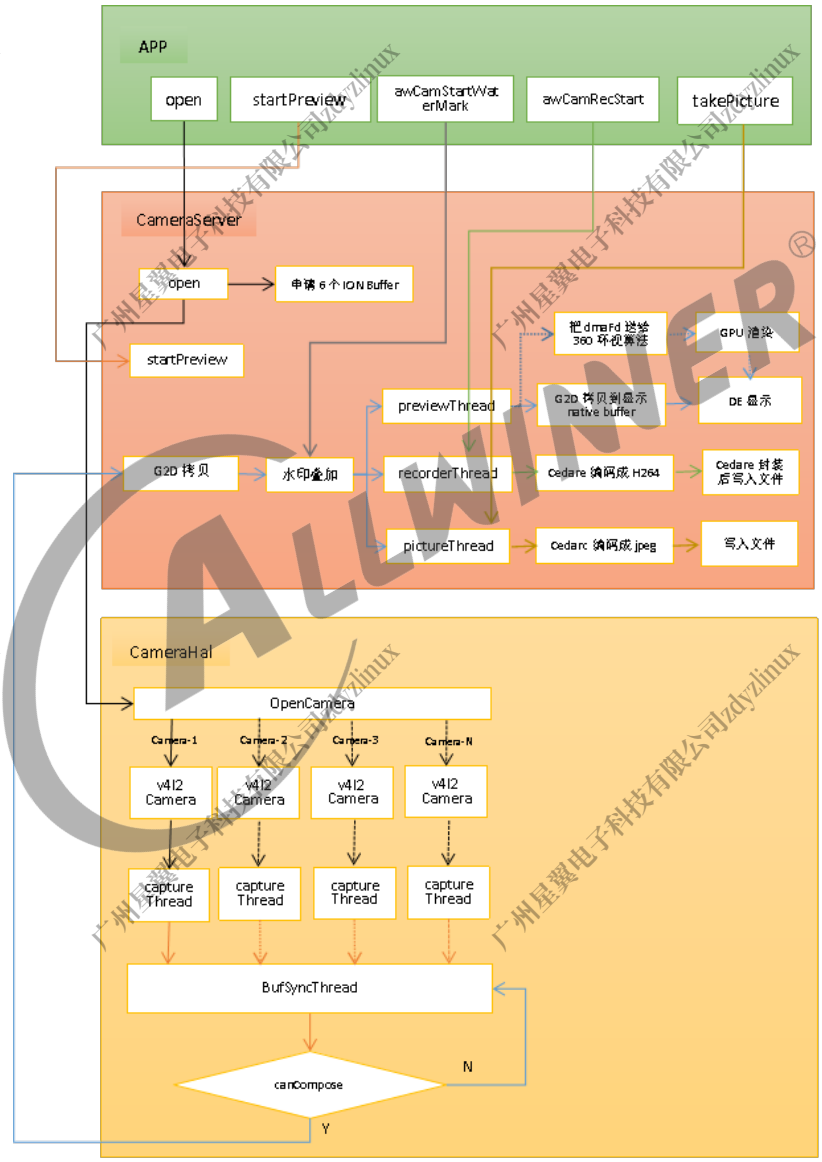

图 6-1: hal1 框架图

1. 对于车载 Camera 特性的实现，均封装在 libawcamera.so 的动态库中，可兼容原生 API1 的预览/拍照/录像等基本功能。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. 对于多路 Camera 同时输入场景，多路 Camera 数据会在 BufSyncTherad 线程中同步，如 4 路 Camera 会等待 4 路 Camera 数据一起送到 awcamera.so 中进行处理，对于等待期间冗余的 buf 则做丢弃旧 buf 处理。
3. recordThread 中兼容 cedarc 编码以及 cedare 编码，cedare 可以支持大小码流。
4. 水印叠加，一般车载上预览、拍照以及录像的画面均需要叠加水印，所以若使能水印功能，需要在拼接后的画面中叠加水印，然后送显示、送编码器。

说明

关于详细开发，可查阅《T527_Android_awCamera_API_ 使用指南》

6.2 Camera Hal3 框架

对于 hal 层而言，主要工作是实现这两个结构体，camera_module，camera3_device_ops_t

6.2.1 camera_module

> camera module 的常规方法，必须是第一个成员变量，因为使用该数据结构的用户会将 hw_module_t 强制转化为 camera_module 指针。

1. ENODEV: camera 设备不能够被打开因为内部错误。
2. EINVAL: 输入参数是无效的。也就是说 id 是无效或者 module 是无效的。
3. EBUSY: 对该 camera id，camera 设备已经调用了 open。
4. EUSERS: 已经到达了最大能够打开 camera 设备的数目了。
5. 其余 common.methods->open 都会返回 ENODEV。
6. 返回 0 时表示成功打开设备。

```
//hardware/aw/camera/hal_3/hal/v4l2_camera_hal.cpp
#define CAMERA_MODULE_API_VERSION_2_4 HARDWARE_MODULE_API_VERSION(2, 4)
#define HARDWARE_HAL_API_VERSION HARDWARE_MAKE_API_VERSION(1, 0)
#define CAMERA_HARDWARE_MODULE_ID "camera"
static hw_module_methods_t v4l2_module_methods = {
  .open = v4l2_camera_hal::open_dev};
...
camera_module_t HAL_MODULE_INFO_SYM __attribute__((visibility("default"))) = {
  .common =
    {
      .tag = HARDWARE_MODULE_TAG,
      .module_api_version = CAMERA_MODULE_API_VERSION_2_4,
      .hal_api_version = HARDWARE_HAL_API_VERSION,
      .id = CAMERA_HARDWARE_MODULE_ID,
      .name = "V4L2 Camera HAL v3",
      .author = "ZJW",
      .methods = &v4l2_module_methods,
      .dso = nullptr,
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
        .reserved = {0},
    },
    .get_number_of_cameras = v4l2_camera_hal::get_number_of_cameras,
    .get_camera_info = v4l2_camera_hal::get_camera_info,
    .set_callbacks = v4l2_camera_hal::set_callbacks,
    .get_vendor_tag_ops = v4l2_camera_hal::get_vendor_tag_ops,
    .open_legacy = v4l2_camera_hal::open_legacy,
    .set_torch_mode = v4l2_camera_hal::set_torch_mode,
    .init = nullptr,
    .reserved = {nullptr, nullptr}};
}
```

V4L2CameraHAL 的接口流程图如下:

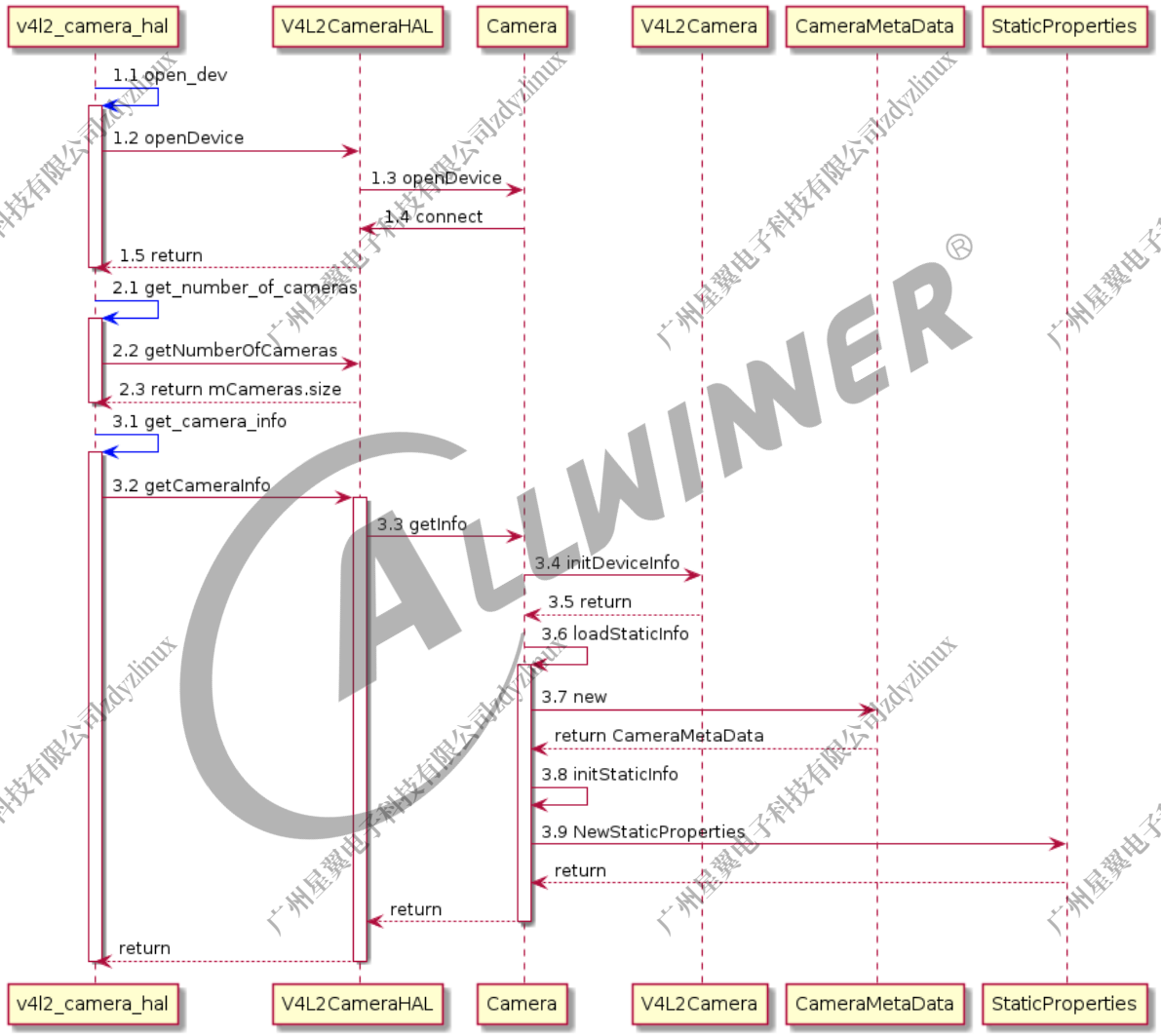

图 6-2: V4L2CameraHAL 接口流程图

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.2.2 camera3_device_ops_t

6.2.2.1 initialize

> initialize 本质是将 framework 的 callback 函数指针传到 HAL 层，该操作必须要在 open() 后且在其他所有函数之前调用。

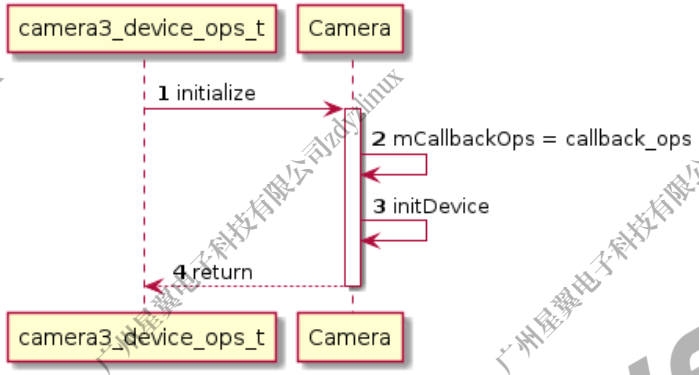

图 6-3: camera3_device_ops_t–initialize 流程图

Camera 的 initialize 主要设置了回调方法 callback_ops，具体回调函数后续展开分析。initDevice 实现暂时为空，为了初始化设备保留的接口。

6.2.2.2 configure_stream

> - configure_stream 会重新设置 Camera HAL 设备的处理流水线 (pipeline) 并设置新的输入出流 (streams), 该调用将覆盖原有 stream_list 中的配置 (stream configuration), 该调用会在 initialize 调用后调用，并且在 processs_capture_requests 提交 request 前调用。

- configure_stream 的参数 stream_list 必须至少包含一个输出流 (output-capble stream), 且可能包含超过一个输入流 (input-capable stream)。
- stream_list 可以包含当前正在使用的流 (通过最近调用 configure_streams 获得的)。这类流将保存有效的 usage 值，max_buffers 以及 private 指针 (camera3_stream_t)。
- 如果 HAL 需要为一个已有的流改变配置 (stream configuration)，它将根据这次 configure_stream 调用重新设置 usage 的值，或者 max_buffers 的值.
- framework 层将能够检测出这些改动，并在这次 request 使用这些 buffers 前，重新分配流的 buffers。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 如果这次的传参 stream_list 并不包含当前使用的流，HAL 层能够安全的移除这些流的所有引用，它们将不会再被 framework 重用。并且所有的 gralloc buffers 将会在 configure_streams 调用返回前被释放。
- 参数 stream_list 是属于 framework 的，并在 configure_streams 完成后不能被访问。camera3_stream_t 的地址在 HAL 层将保持有效，直到第一次不包含 camera3_stream_t 参数的 configure_stream 接口被调用。除了 usage 以及 max_buffers 变量在 configure_streams 调用时能够改动，HAL 层不能修改其他 stream 结构体的值 (private pointer 除外)。
- 如果流是新设置的，结构体中的 max_buffer 以及 private pointer 将会设置为 0。usag 将会被设置为消费者 flags(consumer usage flags)。HAL 设备必须在 configure_streams 返回时设置这些变量。这些变量将会被 Framework 以及平台的 gralloc 模块用于为每一个流分配 gralloc 内存。
- 新分配的 buffers 可能会被 framework 包含在一个 capture request 中。一旦 gralloc buffers 通过回调方法 process_capture_result 返回到 framework,framework 就有可能在任何时间内立即释放内存或者重新使用它。

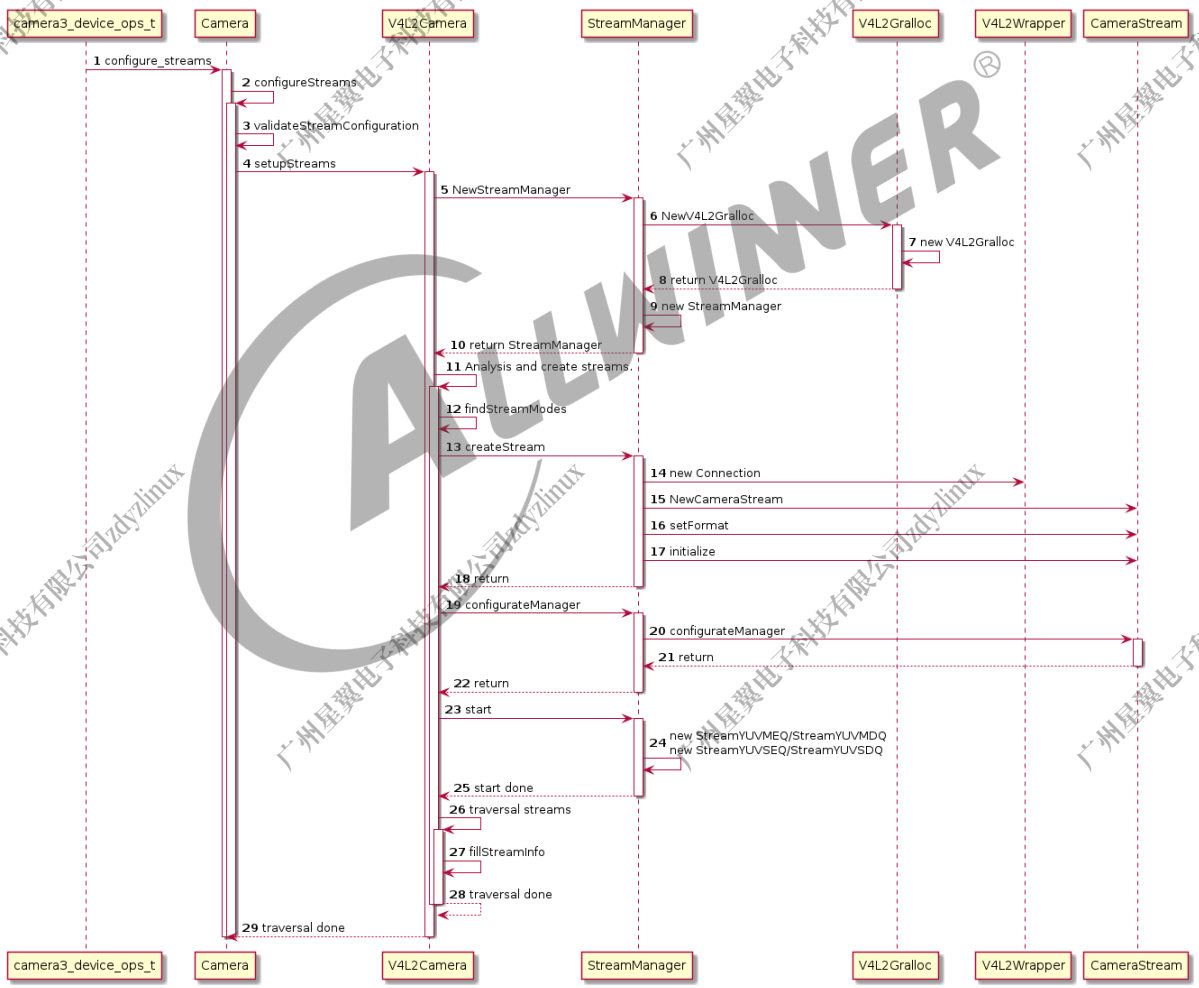

图 6-4: configure_stream

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.2.2.3 construct_default_request_settings

> construct_default_request_settings 为通用的 Camera 创建 capture 设置，设备必须返回一个 settings buffer 满足配置要求，并且为 CAMERA3_TEMPLATE_* 的其中一项。HAL 层拥有该结构体，并且指向的指针必须在设备关闭前保持有效。framework 和 HAL 在 construct_default_request_settings 返回前可能不会进行修改。

6.2.3 数据流向

6.2.3.1 process_capture_request

> process_capture_request 发送一个新的 capture 请求到 HAL。HAL 直到准备好接受下一个 capture 请求前，都不会返回该方法。framework 同一时内只会调用一次 process_capture_request，且调用只会来自同一个线程。下一次使用 process_capture_request 作为一次新的请求会在其对应的 buffers 准备好时调用。在预览的情境下，这意味着方法会被 framework 立即调用。

> 在整个 process_capture_request 的调用中，从 HAL 层获取 capture 结果的流程是异步的。该调用要求 metadata 必须是有效的，但输出的 buffer 可能会提供 sync fences 用于等待。多个的 request 期望能够同时传输，以保持完整的输出帧率。

> framework 拥有 request 结构权。这只能够保证在本次调用中是该 request 请求是有效的。HAL 层设备必须拷贝这些需要的信息以用于 capture 的处理。HAL 层有责任去等待以及关闭 buffer 的 fences 并且返回 buffer 的 handle 到 framework。

> 假如 input_buffer 不为空，HAL 必须将输入 buffer 的 release sync fence 的文件的描述符写入到 input_buffer->release_fence. 假如 HAL 关于 input buffer 的 release sync fence 返回-1，framework 就能够立即重用这些 input buffer。否则 Framework 将会等待 sync fence 才会去重新填充或者重用输入 buffer。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

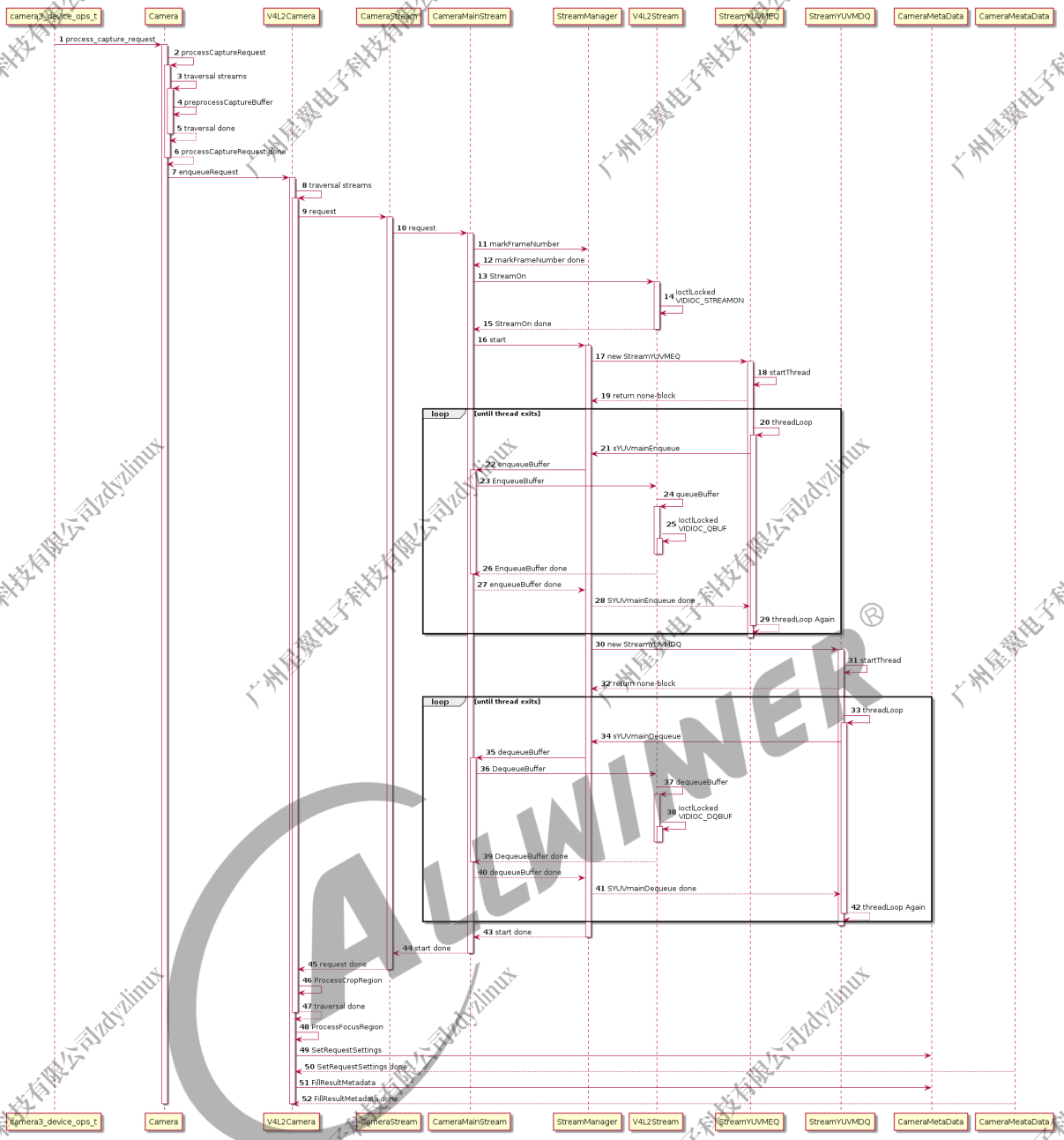

图 6-5: process_capture_request

6.2.3.2 process_capture_result

发送处理结果到 framework。单次的 capture 请求，可能会被 HAL 层调用多次 process_capture_result。这样的设计能够允许 metadata 以及低分辨率的 buffer 立刻能够被返回，后处理 (post-processed) 的 JPEG buffer 能够在后续准备好后进行调用。每次调用必须包括请求的帧序号以对应 metadata 或者 buffers。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

在 process_capture_result 调用时，可能只会包含一个 component(buffer 或者 metadata), 每一个 stream 的 buffer 以及 result metadata，在每次 process_capture_result 对应单个请求时，必须被 HAL 返回 (输出时产生错误时也会返回结果)。不允许一个既不包含 output buffer 或者 result metadata 的 process_capture_result 调用被执行。

在同一结果中 metadata 和 buffers 的顺序并不影响。但对于指的 stream 中，buffers 必须是遵循 FIFO 的顺序。stream A 的 request 5 的返回必须要先于 request 6。同理 metadata 也一样。

然而不同的流之间独立的，所以 stream A 的 request 5 可能会比 stream B 的 request 6 返回得慢。metadata 同理也一样。

HAL 层保留 result 结构的拥有权，只需要保证该数据在本次调用有效即可。framework 将会在调用返回前拷贝它所需要的内容。

output buffers 不需要被填充。framework 会在等待流缓冲区的 release sync fence 前读取所有的缓冲区数据。因此该方法应该被 HAL 层尽可能快，尽管部分 output buffer 还可能仍在填充。HAL 必须包括有效的 release sync fences 填充到每个流的 output_buffers entry，或者返回-1 如果 stream buffer 已经被填满了。

如果 result buffer 不能够根据一个 request 被构建，HAl 应该返回一个空的 metadata buffer，但仍需要提供输出 buffers 以及他们的 sync fences。并且 notify 方法必须发送 ERROR_RESULT 信息。

假如 output buffer 不能够被填满，起 status 字段必须设置为 STATUS_ERROR。并且调用 notify 方法发送 ERROR_BUFFER 信息。

如果整个 capture 失败了，那么该方法仍然需要被调用以返回 output buffers 到 framework。所有的 buffer 状态需要设置为 STATUS_ERROR，并且 result metadata 应该为 empty buffer。并且 notify 需要发送 ERROR_REQUEST 信息。在这种情境下，单独的 ERROR_RESULT/ERROR_BUFFER 不应该发送。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

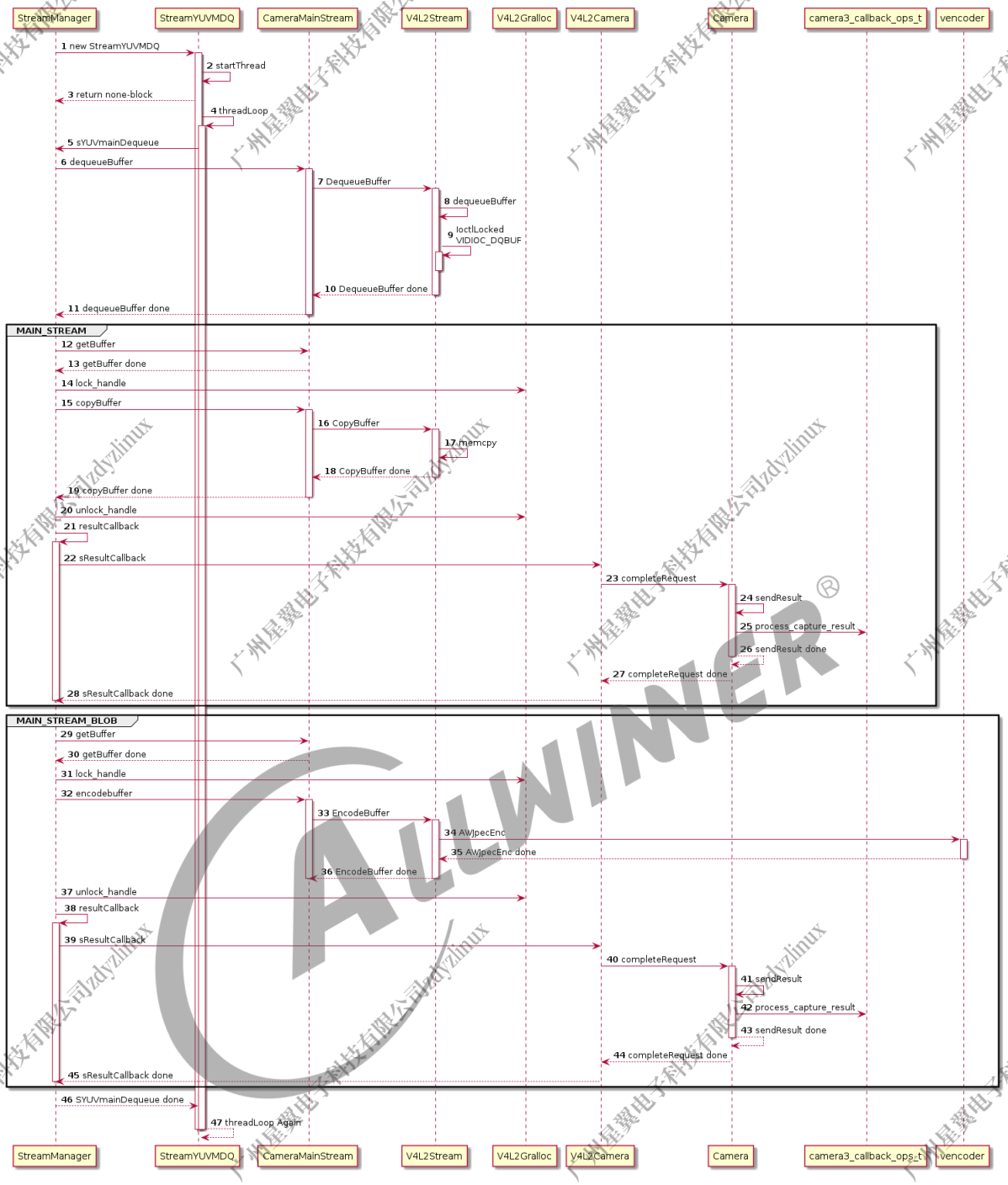

图 6-6: process_capture_result

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7 Camera 配置

7.1 内核配置介绍

内核相关配置可以查看《Linux_MIPI_CSI_ 开发指南》。

7.2 Sensor 自适应配置

多 sensor 自适应配置可以查看《Android_13_Camera 自动检测 _ 使用指南》。

7.3 HAL 选择

以下配置均需在方案目录下的 device-common.mk 中配置

车载方案一般使用 hal1。

```
USE_CAMERA_HAL_AUTO_1_0 :=true
```

非车载方案则使用 hal_3，有以下配置：

```
#如需使用CSI Camera则使能，否则关闭
USE_CAMERA_HAL_3_4 := true
#如需使用USB Camera则使能，否则关闭
PRODUCT_HAS_UVC_CAMERA := true
```

7.4 CSI Camera 配置

7.4.1 配置文件介绍

在安卓的方案目录下，提供一下几个文件对 Camera 进行配置：

init.camera.rc

Camera 驱动加载定义在 android/device/softwinner/[方案]/[样机]/camera/init.camera.rc 中实现，加载顺序如下，且需保证 vin_v4l2.ko 在最后进加载。如下所示；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
on early-boot
### csi module
    insmod /vendor/lib/modules/vin_io.ko
    insmod /vendor/lib/modules/s5k5e8.ko
    insmod /vendor/lib/modules/imx319_mipi.ko
    insmod /vendor/lib/modules/vin_v4l2.ko
```

camera.cfg

Camera.cfg 用于对 Camera 功能进行配置，包括自动对焦，闪光灯，变焦等功能；路径位于 SDK 的 device/softwinner/[方案]/[样机]/camera/configs 下。也可以直接通过 adb push 到设备目路/vendor/etc/中，重启即可生效。

media_profiles.xml

media_profiles.xml 主要用于保存 Camera ID 对应支持的摄像相关参数，包括摄像质量，音视频编码格式，帧率，比特率等。

说明

切记在 media_profiles.xml 中申明的分辨率，都需要在 camera.cfg 中申明。

7.4.2 Camera 方向配置

Android 方向配置可以在方案目录的 camera.cfg 中进行配置，如下所示：

```
;-----------------------------------------------------------------------
; camera orientation (0, 90, 180, 270)
;-----------------------------------------------------------------------
camera_orientation = 0
```

驱动层的水平和垂直镜像翻转可以通过修改 dts 去，路径在 device/config/chips/[ic name]/configs/[borad name]/board.dts, 找到对应的 sensor flip 进行修改，如 sensor0。

```
sensor0_hflip        = 1
sensor0_vflip        = 1
```

7.4.3 数字变焦功能配置

Android 变焦配置可以在方案目录的 camera.cfg 中进行配置，如下所示：

```
used_zoom = 1 //0表示关闭数字变焦功能，1表示打开数字变焦功能
key_max_zoom = 2 //最大变焦倍数。最大变焦倍数不能超过（sensor出的最大分辨率/预览分辨率）的倍数
key_default_zoom = 0 //开机默认放大倍数。如果放大1.2倍，则配置成key_default_zoom = 120. 默认放大倍数不可超过最大
    放大倍数。
```

7.4.4 单摄配置

对于需要配置单摄像头的客户，可直接在 camera.cfg 中进行配置：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
number_of_camera = 1（配置为单摄）
camera_facing = 1（1：表示配置为前摄 0：表示配置为前摄）
```

7.4.5 插值方案配置

在应用场景中有客户需要配置低分辨率的 sensor 拍照的时候出更高分辨率的图像；如 30w 插值到 120w。

camera.cfg 中, 如下为配置将 30w（640x480）插值到 120w(1280x960)，并添加 interpolation 配置。

```
used_picture_size = 1
key_support_picture_size = 1280x960,640x480,320x240
key_default_picture_size = 1280x960

used_interpolation_size = 1
key_support_src_interpolation_size = 640x480
key_default_dst_interpolation_size = 1280x960
```

说明

1. **camera.cfg 需支持 media_profile.xml 中的所有分辨率，不然会出现录像异常。**
2. **如果客户需要插值到 1300W 的时候, 建议可以配置为 4096x3200。**
3. **在美颜拍照的时候出现闪屏问题条纹，一般为带宽不够，可尝试提高 dram 频率或降低屏幕刷新速度进行改善。**
4. **在配置 media_profile 的时候也注意，不要超过 sensor 本身能支持的分辨率且分辨率不得高于 1080P。：**

7.4.6 闪光灯

1. 配置内核

步骤一：./build.sh menuconfig ，配置 use flash module。

```
> Allwinner BSP -> Device Drivers ->VIN (camera) Drivers -><M> use flash module
```

步骤二：dts 打开闪光灯配置。可查阅《Linux_MIPI_CSI_ 开发指南》中 Device Tree 配置说明章节配置 flash 模块，如下所示：

```
flash0: flash@2108190 {
    device_type = "flash0";
    flash0_type = <2>;
    flash0_en = <&r_pio PL 11 GPIO_ACTIVE_LOW>;
    flash0_mode = <>;
    flash0_flvdd = "";
    flash0_flvdd_vol = <>;
    device_id = <0>;
    status = "okay";
};
```

步骤三：sensor 驱动检查。需要添加 VIDIOC_VIN_FLASH_EN，对 flash 进行控制，如下所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
static long sensor_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
{
    int ret = 0;
    struct sensor_info *info = to_state(sd);

    switch (cmd) {
    case GET_CURRENT_WIN_CFG:
        if (info->current_wins != NULL) {
            memcpy(arg, info->current_wins,
                sizeof(*info->current_wins));
            ret = 0;
        } else {
            sensor_err("empty wins!\n");
            ret = -1;
        }
        break;
    case SET_FPS:
        ret = 0;
        break;
    case VIDIOC_VIN_SENSOR_EXP_GAIN:
        ret = sensor_s_exp_gain(sd, (struct sensor_exp_gain *)arg);
        break;
    case VIDIOC_VIN_SENSOR_CFG_REQ:
        sensor_cfg_req(sd, (struct sensor_config *)arg);
        break;
    case VIDIOC_VIN_FLASH_EN:
        ret = flash_en(sd, (struct flash_para *)arg);
        break;
    default:
        return -EINVAL;
    }
    return ret;
}
```

图 7-1: sensor 驱动检查

2. 配置 Android

步骤一：添加 flash 特性。

```
#android/device/softwinner/common/config/tablet_core_hardware.xml
<feature name="android.hardware.camera.flash" />
```

步骤二：camera.cfg 使能 flash。

```
used_flash_mode = 1
```

7.4.7 对焦功能配置

1. 内核配置

步骤一：确认内核 deconfig 配置是否打开。

```
> Allwinner BSP -> Device Drivers ->VIN (camera) Drivers ->  use flash module   -> <M> use actuatormodule
```

步骤二：dts 打开相关配置。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

可查阅《Linux_MIPI_CSI_ 开发指南》中 Device Tree 配置说明章节；使能对焦马达；以 dw9714_act 为例：

```
actuator0: actuator@2108180 {
    device_type = "actuator0";
    actuator0_name = "dw9714_act";
    actuator0_slave = <0x18>;
    actuator0_af_pwdn = <>;
    actuator0_afvdd = "afvcc-csi";
    actuator0_afvdd_vol = <2800000>;
    status = "okay";
};
```

2.Android 配置

步骤一：添加对焦特性。

```
#android/device/softwinner/common/config/tablet_core_hardware.xml
<feature name="android.hardware.camera.autofocus" />
```

步骤二：方案中的 init.camera.rc 需加载对应驱动马达的驱动。

```
on early-boot
### csi module
    insmod /vendor/lib/modules/vin_io.ko
    insmod /vendor/lib/modules/s5k5e8.ko
    insmod /vendor/lib/modules/imx319_mipi.ko
    insmod /vendor/lib/modules/actuator.ko  #添加马达驱动
    insmod /vendor/lib/modules/dw9714_act.ko  #添加dc9714对应的控制驱动
    insmod /vendor/lib/modules/vin_v4l2.ko
```

步骤三：camera.cfg 使能对焦功能。

```
used_focus_mode = 1
```

7.4.8 Camera 节点配置

当前 hal 默认使用 video0，video4 对应的 camera0,video8,video12 对应的 Camera1,video2,video6 对应的是 camera2, 在使用过程中，如需动态修改 video 节点，camera.cfg 可以按照如下配置；

```
device_id = 0
+camera_video_node = 0,4
```

7.5 USB Camera 配置

7.5.1 内核配置

全志的 USB 通路利用的是谷歌原生的 UVC Hal，有一些耗时的操作走 VE 流程，内核 defconfig 已默认打开, 客户只需关注 dts 和方案下面的配置。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

dts 配置：如果只是使用 USB Camera,DTS 中对 csi sensor 的配置需 disable 掉。

7.5.2 Andoird 配置

UVC 配置文件：

```
SDK/device/softwinner/[platform_name]/[board_name]/
├── device-commin.mk
├── camera
│   ├── external_camera_config.xml
│   └── media_profiles.xml //配置支持的相关录像格式
```

external_camera_config.xml 文件：配置 USB Camera 相关配置，CameraID, 忽略的 Video 节点，帧率，以及 Camera 方向。

```
<ExternalCamera>
    <Provider>
    <!-- Internal video devices to be ignored by external camera HAL -->
    <!-- 如果产品带有CSI摄像头，如产品中有两个CSI摄像头，则CameraIdOffset 填-2,igonre代表CSI对应的video节点，填写
        后UVC进程会自动过滤相关节点不打开
    -->
    <!--
    <CameraIdOffset>-2</CameraIdOffset>
        <ignore>
            <id>0</id>
            <id>1</id>
            <id>2</id>
            <id>3</id>
        </ignore>
    -->
    <!-- Not iternal video devices to be ignored by external-only camera HAL -->
    <!-- 如果产品无CSI摄像头-->
    <CameraIdOffset>0</CameraIdOffset>
        <ignore>
        </ignore>
    </Provider>
    <!-- See ExternalCameraUtils.cpp for default values of Device configurations below -->
    <Device>
        <!-- Max JPEG buffer size in bytes-->
        <MaxJpegBufferSize bytes="3145728"/> <!-- 3MB (~= 1080p YUV420) -->
        <!-- Size of v4l2 buffer queue when streaming >= 30fps -->
        <!-- Larger value: more request can be cached pipeline (less janky) -->
        <!-- Smaller value: use less memory -->
        <NumVideoBuffers count="4"/>
        <!-- Size of v4l2 buffer queue when streaming < 30fps -->
        <NumStillBuffers count="2"/>
        <!-- List of maximum fps for various output sizes -->
        <!-- Any image size smaller than the size listed in Limit row will report
            fps (as minimum frame duration) up to the fpsBound value. -->
        <FpsList>
            <!-- width/height must be increasing, fpsBound must be decreasing-->
            <Limit width="640" height="480" fpsBound="30.0"/>
            <Limit width="1280" height="720" fpsBound="15.0"/>
            <!-- image size larger than the last entry will not be supported-->
        </FpsList>
        <CfgList>
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
        <!-- 根据实际的cameraId与朝向配置对应的degree(sensorOrientation),facing以及jpegOrientation-->
        <CameraCfg cameraId="1" degree="0" facing="1" jpegOrientation="0"/>
        <CameraCfg cameraId="2" degree="0" facing="0" jpegOrientation="0"/>
        <CameraCfg cameraId="3" degree="0" facing="1" jpegOrientation="0"/>
    </CfgList>
  </Device>
</ExternalCamera>
```

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

8 常见问题排查

此部分可以查阅《Android_Camera 量产问题 _ 排查指南》。

广州星翼电子科技有限公司zdyzlinux

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。